

[Наименование образовательной организации]

[Кафедра / отделение]

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту

на тему

**«Разработка мобильного приложения для аренды спортивного
инвентаря»**

Выполнил: _____

Группа: _____

Руководитель: _____

Город: _____

[Год]

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ.....	3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ АСПЕКТЫ ИЗУЧАЕМОЙ ПРОБЛЕМЫ	5
1.1 Характеристика предметной области	5
1.2 Требования к цифровому сервису аренды	5
1.3 Обоснование выбора средств разработки	6
ГЛАВА 2. ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ПРИЛОЖЕНИЯ	8
2.1 Проектирование базы данных.....	8
2.2 Проектирование архитектуры решения	22
ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ.....	24
3.1 Реализация основных пользовательских сценариев.....	24
3.2 Разработка интерфейса приложения.....	24
3.3 Тестирование и проверка работоспособности.....	29
3.3.1 Сборка локальной базы данных.....	29
3.3.2 Проверка пользовательских сценариев	29
3.3.3 Ограничения текущей версии.....	30
ГЛАВА 4. РАСЧЕТ ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ ПРОЕКТА	31
4.1 Анализ затрат на разработку.....	31
4.2 Оценка экономической эффективности	32
ЗАКЛЮЧЕНИЕ	34
СПИСОК ЛИТЕРАТУРЫ	35

ВВЕДЕНИЕ

Актуальность разработки приложений для аренды спортивного инвентаря определяется ростом спроса на краткосрочное пользование оборудованием без необходимости его покупки. Для клиента важно быстро найти нужную позицию, проверить доступность в конкретном пункте проката и получить понятные условия аренды.

Тема дипломного проекта связана с созданием мобильного приложения на базе кроссплатформенной технологии .NET MAUI, объединяющего проектирование базы данных и реализацию пользовательского интерфейса. В рамках проекта SportRent реализован автономный демонстрационный сценарий, в котором основные действия выполняются на базе локальной SQLite без серверной части.

Целью работы является разработка мобильного приложения SportRent, обеспечивающего авторизацию пользователя, просмотр каталога, получение сведений об инвентаре, оформление аренды, просмотр истории заказов и личного кабинета.

Работа выполнена на примере учебного сервиса проката спортивного инвентаря SportRent, моделирующего деятельность пункта проката с локальной базой данных и типовыми пользовательскими сценариями.

В дипломной версии пояснительной записки основной акцент сделан на мобильном сценарии использования, практической значимости решения и завершенности пользовательского маршрута аренды в кроссплатформенном приложении .NET MAUI.

Для достижения поставленной цели были решены следующие задачи:

1. Выполнен анализ предметной области аренды спортивного инвентаря.
2. Спроектирована нормализованная структура реляционной базы данных.

3. Реализован инструмент автоматической сборки SQLite из SQL-скриптов.

4. Создано кроссплатформенное мобильное приложение на платформе .NET MAUI.

5. Проверена работоспособность ключевых пользовательских сценариев.

6. Выполнен учебный расчет экономической эффективности проекта.

Объектом исследования является процесс аренды спортивного инвентаря, а предметом исследования выступают методы проектирования локальной базы данных и реализации мобильного приложения для обслуживания данного процесса.

В работе использованы методы анализа предметной области, сравнения аналогичных решений, проектирования структуры данных, программной реализации, тестирования и условного экономического расчета.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ АСПЕКТЫ ИЗУЧАЕМОЙ ПРОБЛЕМЫ

1.1 Характеристика предметной области

Предметная область включает пользователей, пункты проката, категории и конкретные позиции инвентаря, тарифы аренды, остатки, заказы и платежи. Для одного и того же вида инвентаря необходимо хранить сведения о доступности в разных пунктах, размерах и состояниях, а также учитывать залог и срок аренды.

В текущей версии проекта в локальной базе данных хранится 5 категорий, 5 позиций инвентаря и 3 пункта проката. Пользователю доступны такие примеры оборудования, как Горный велосипед, Лыжный комплект, Сноуборд, Электросамокат, Шлем защитный. Эти данные достаточны для демонстрации типового потока работы приложения на защите.

Ключевая особенность рассматриваемой предметной области состоит в необходимости связать каталог с реальными остатками инвентаря. Пользователь должен видеть не только карточку товара, но и доступность выбранной позиции в конкретном пункте проката, а также применимые тарифы и размер залога.

1.2 Требования к цифровому сервису аренды

На основании анализа предметной области были сформулированы требования к программному средству. Пользовательская часть должна обеспечивать быстрый вход в систему, поиск и фильтрацию каталога, просмотр детальной карточки, оформление аренды и получение информации о ранее созданных заказах.

Система должна работать в автономном режиме, поскольку проект ориентирован на учебную демонстрацию без зависимости от удаленной инфраструктуры. Поэтому база данных, изображения инвентаря и логика доступа к данным размещены локально в составе клиента.

Дополнительным требованием стало обеспечение воспроизводимости данных. Для этого в решении выделен отдельный инструмент сборки базы данных из канонических SQL-скриптов и подготовлены демонстрационные учетные записи разных ролей.

Для мобильной версии дополнительно важны компактная компоновка элементов, вертикальная прокрутка контента и удобство сенсорного взаимодействия на экранах ограниченного размера.

1.3 Обоснование выбора средств разработки

Для реализации проекта выбран стек, включающий .NET 9 в качестве платформы, .NET MAUI для построения интерфейса, язык C#, разметку XAML, СУБД SQLite и Entity Framework Core 9 для слоя данных. Такой набор средств соответствует теме дипломного проекта и позволяет объединить разработку интерфейса, локального хранения и прикладной логики в одной технологии.

Выбор SQLite обусловлен малым объемом демонстрационных данных, простотой развертывания и возможностью хранить файл базы непосредственно в составе приложения. .NET MAUI обеспечивает кроссплатформенную основу и единый стек для реализации страниц, навигации и взаимодействия с локальными ресурсами.

Таблица 1

Использованные технологии и их назначение

Технология	Назначение
.NET 9	Платформа разработки и выполнения проекта
.NET MAUI	Построение кроссплатформенного пользовательского интерфейса

C#	Реализация прикладной логики и сервисов
XAML	Описание экранов приложения
SQLite	Локальное хранение данных
Entity Framework Core 9	Scaffolded слой данных и модели
PowerShell	Автоматизация сборки базы данных и scaffold-процесса

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СТРУКТУРЫ ПРИЛОЖЕНИЯ

2.1 Проектирование базы данных

База данных проекта сформирована как нормализованная реляционная структура и в текущем состоянии содержит 20 таблиц. При проектировании отдельно выделены справочники ролей, категорий, брендов, типов аренды, статусов заказов и платежей, что исключает дублирование данных и упрощает дальнейшее расширение системы.

Основной сущностью каталога является таблица `equipment`, содержащая сведения о конкретной позиции инвентаря. Тарифы вынесены в отдельную таблицу `equipmentRates`, что позволяет задавать несколько вариантов аренды для одной позиции. Остатки хранятся в `rentalPointEquipment` и связаны одновременно с пунктом проката, размером и состоянием оборудования.

Для корректного хранения медиафайлов реализована нормализация изображений через таблицы `images` и `equipmentPhotos`. Позиции заказа связаны не просто с инвентарем вообще, а с конкретным остатком из `rentalPointEquipment`, благодаря чему приложение может корректно уменьшать доступное количество при создании заказа.

Таблица 2

Ключевые группы таблиц базы данных

Группа	Содержание
Пользователи и роли	roles, users
Каталог и классификаторы	categories, brands, equipmentTypes, equipmentSizes, equipmentConditions
Прокат и доступность	rentalTypes, rentalPoints, equipment, equipmentRates, rentalPointEquipment

Изображения	images, equipmentPhotos
Заказы и статусы	orderStatuses, rentOrders, orderItems
Платежи	paymentMethods, paymentStatuses, payments

Важным проектным решением стало хранение денежных значений в целочисленном формате, то есть в копейках. Это исключает ошибки округления и делает операции расчета аренды и залога более предсказуемыми.

На рисунке 1 представлена логическая схема базы данных SportRent. На ней показаны основные таблицы и внешние ключи между пользователями, каталогом, остатками, заказами и платежами.

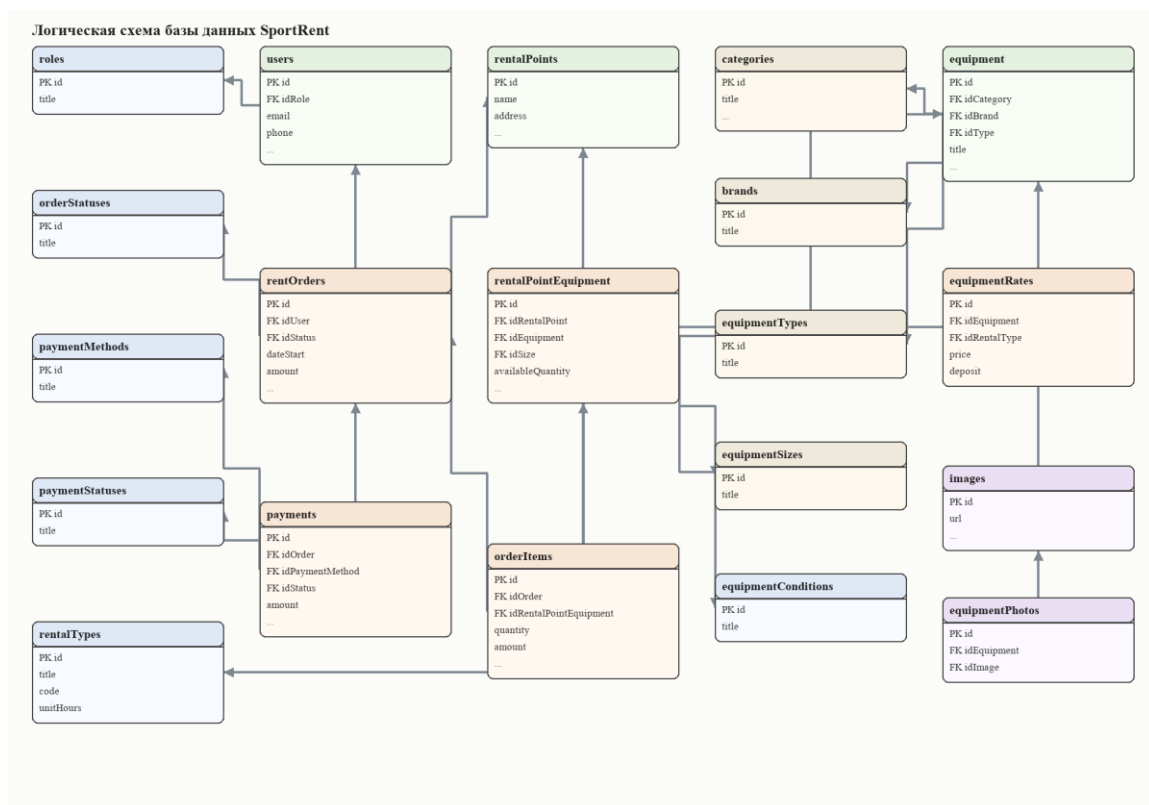


Рис. 1. Логическая схема базы данных SportRent

Схема показывает, что таблица equipment образует центр каталожного контура, rentalPointEquipment связывает каталог с фактическими остатками по

точкам проката, а rentOrders, orderItems и payments формируют транзакционный контур аренды. Таблица rentOrders дополнительно содержит две ссылки на rentalPoints, что позволяет отдельно хранить пункт выдачи и пункт возврата инвентаря.

Подробное описание полей приведено ниже. Наименования таблиц и колонок сохранены в том виде, в котором они реализованы в физической модели SQLite и используются прикладным кодом.

Справочники и классификаторы базы данных

Таблица 3

Структура таблицы roles: справочник ролей пользователей

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Таблица 4

Структура таблицы orderStatuses: справочник статусов заказов аренды

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Таблица 5

Структура таблицы paymentMethods: справочник способов оплаты

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Таблица 6

Структура таблицы paymentStatuses: справочник статусов платежей

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Таблица 7

Структура таблицы equipmentConditions: справочник состояний инвентаря

Поле	Тип	Назначение	Ограничения и связи
------	-----	------------	------------------------

id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Таблица 8

Структура таблицы rentalTypes: варианты тарификации аренды и длительность периода

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL
code	TEXT	Краткий символьный код справочного значения.	UNIQUE; NOT NULL
unitHours	INT	Продолжительность ь одного тарифного интервала в часах.	NOT NULL

Таблица 9

Структура таблицы categories: категории спортивного инвентаря

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK

title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL
description	TEXT	Развернутое описание категории инвентаря.	Без специальных ограничений.

Таблица 10

Структура таблицы brands: справочник брендов оборудования

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Таблица 11

Структура таблицы equipmentTypes: классификатор типов инвентаря

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Таблица 12

Структура таблицы equipmentSizes: классификатор размеров инвентаря

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
title	TEXT	Текстовое наименование сущности или справочного значения.	UNIQUE; NOT NULL

Операционные сущности базы данных

Таблица 13

Структура таблицы rentalPoints: пункты выдачи и возврата инвентаря

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
name	TEXT	Наименование пункта проката.	NOT NULL
address	TEXT	Почтовый адрес объекта.	NOT NULL
phone	TEXT	Контактный телефон.	Без специальных ограничений.

Таблица 14

Структура таблицы users: учетные записи клиентов и сотрудников

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
idRole	INT	Ссылка на роль пользователя.	FK -> roles.id; NOT NULL
firstName	TEXT	Имя пользователя.	NOT NULL
lastName	TEXT	Фамилия пользователя.	NOT NULL
email	TEXT	Адрес электронной почты пользователя.	UNIQUE; NOT NULL
phone	TEXT	Контактный телефон пользователя.	UNIQUE; NOT NULL
passwordHash	TEXT	Пароль или его строковое представление, используемое в демонстрационной авторизации.	NOT NULL
dateCreated	TEXT	Дата и время создания записи.	NOT NULL; DEFAULT CURRENT_TIMESTA MP

Таблица 15

Структура таблицы equipment: карточки позиций спортивного инвентаря

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK

idCategory	INT	Ссылка на категорию инвентаря.	FK -> categories.id; NOT NULL
idBrand	INT	Ссылка на бренд оборудования.	FK -> brands.id; NOT NULL
idType	INT	Ссылка на тип инвентаря.	FK -> equipmentTypes.id; NOT NULL
title	TEXT	Наименование позиции инвентаря, отображаемое в каталоге.	NOT NULL
description	TEXT	Описание характеристик и особенностей позиции инвентаря.	Без специальных ограничений.
model	TEXT	Модель или артикул позиции инвентаря.	Без специальных ограничений.

Таблица 16

Структура таблицы equipmentRates: тарифы аренды и залог по позициям инвентаря

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
idEquipment	INT	Ссылка на позицию инвентаря.	FK -> equipment.id, DELETE CASCADE; NOT NULL
idRentalType	INT	Ссылка на тип тарифа аренды.	FK -> rentalTypes.id; NOT NULL

price	INT	Стоимость аренды за выбранный тарифный период.	NOT NULL
deposit	INT	Сумма залога по позиции инвентаря.	NOT NULL; DEFAULT 0

Для таблицы equipmentRates дополнительно заданы составные ограничения уникальности: UNIQUE(idEquipment, idRentalType).

Таблица 17

Структура таблицы rentalPointEquipment: остатки инвентаря по пунктам проката, размерам и состояниям

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
idRentalPoint	INT	Ссылка на пункт проката.	FK -> rentalPoints.id; NOT NULL
idEquipment	INT	Ссылка на позицию инвентаря.	FK -> equipment.id; NOT NULL
idEquipmentCondition	INT	Ссылка на состояние инвентаря.	FK -> equipmentConditions.id; NOT NULL
idSize	INT	Ссылка на размер инвентаря.	FK -> equipmentSizes.id; NOT NULL
totalQuantity	INT	Общее количество единиц инвентаря в точке проката.	NOT NULL

availableQuantity	INT	Количество единиц, доступных для выдачи в аренду.	NOT NULL
-------------------	-----	---	----------

Для таблицы rentalPointEquipment дополнительно заданы составные ограничения уникальности: UNIQUE(idRentalPoint, idEquipment, idEquipmentCondition, idSize).

Таблица 18

Структура таблицы images: локальные графические ресурсы приложения

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
url	TEXT	Путь к изображению, встроенному в приложение или поставляемому вместе с ним.	UNIQUE; NOT NULL
dateCreated	TEXT	Дата и время создания записи.	NOT NULL; DEFAULT CURRENT_TIMESTAMP

Таблица 19

Структура таблицы equipmentPhotos: связь между позициями инвентаря и изображениями

Поле	Тип	Назначение	Ограничения и связи
------	-----	------------	---------------------

id	INT	Первичный ключ записи.	PK
idEquipment	INT	Ссылка на позицию инвентаря.	FK -> equipment.id, DELETE CASCADE; NOT NULL
idImage	INT	Ссылка на изображение.	FK -> images.id, DELETE CASCADE; NOT NULL

Для таблицы equipmentPhotos дополнительно заданы составные ограничения уникальности: UNIQUE(idEquipment, idImage).

Таблица 20

Структура таблицы rentOrders: заголовки заказов аренды

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
idUser	INT	Ссылка на пользователя, оформившего заказ.	FK -> users.id; NOT NULL
idStatus	INT	Ссылка на текущий статус заказа аренды.	FK -> orderStatuses.id; NOT NULL
idRentalPointIssue	INT	Ссылка на пункт выдачи инвентаря.	FK -> rentalPoints.id; NOT NULL
idRentalPointReturn	INT	Ссылка на пункт возврата инвентаря.	FK -> rentalPoints.id
dateCreated	TEXT	Дата и время создания записи.	NOT NULL; DEFAULT

			CURRENT_TIMESTAMP
dateStart	TEXT	Дата и время начала аренды.	NOT NULL
dateEnd	TEXT	Дата и время завершения аренды.	NOT NULL
amount	INT	Итоговая стоимость аренды без учета залога.	NOT NULL; DEFAULT 0
depositAmount	INT	Суммарный залог по заказу.	NOT NULL; DEFAULT 0
description	TEXT	Комментарий к заказу аренды.	Без специальных ограничений.

Таблица 21

Структура таблицы orderItems: позиции заказов аренды

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
idOrder	INT	Ссылка на заказ аренды.	FK -> rentOrders.id, DELETE CASCADE; NOT NULL
idRentalPointEquipment	INT	Ссылка на конкретный остаток инвентаря в пункте проката.	FK -> rentalPointEquipment.id; NOT NULL
quantity	INT	Количество единиц в составе заказа.	NOT NULL; DEFAULT 1

pricePerUnit	INT	Цена одной единицы в составе заказа.	NOT NULL
amount	INT	Стоимость конкретной позиции заказа.	NOT NULL

Таблица 22

Структура таблицы payments: платежные записи по заказам

Поле	Тип	Назначение	Ограничения и связи
id	INT	Первичный ключ записи.	PK
idOrder	INT	Ссылка на заказ аренды.	FK -> rentOrders.id, DELETE CASCADE; NOT NULL
idPaymentMethod	INT	Ссылка на способ оплаты.	FK -> paymentMethods.id; NOT NULL
idStatus	INT	Ссылка на текущий статус платежа.	FK -> paymentStatuses.id; NOT NULL
dateCreated	TEXT	Дата и время создания записи.	NOT NULL; DEFAULT CURRENT_TIMESTAMP
amount	INT	Сумма платежа по заказу.	NOT NULL

Представленная детализация фиксирует не только перечень сущностей, но и состав их полей, типы данных, обязательность заполнения и внешние ключи. Это делает описание базы данных в записке согласованным с

фактической SQLite-схемой проекта и удобным для последующей доработки приложения.

2.2 Проектирование архитектуры решения

Решение состоит из трех логически разделенных проектов. Такое разделение позволяет обособить задачи построения базы данных, работы со scaffolded сущностями и разработки пользовательского интерфейса.

В клиентском приложении используется упрощенный вариант архитектурного подхода MVVM: страницы отвечают за визуальную часть, ViewModel управляют состоянием экрана, а сервисы инкапсулируют доступ к данным и прикладную логику. Такой подход уменьшает связанность кода и делает сценарии загрузки каталога, авторизации и заказов более прозрачными.

Файл SportRent.Mobile.csproj подтверждает кроссплатформенный характер решения: проект ориентирован на Android, iOS, Mac Catalyst и Windows. Общая XAML-разметка и единый слой сервисов обеспечивают использование одной кодовой базы для разных целей сборки.

В рамках данной версии записки основной акцент сделан на мобильных пользовательских сценариях и переносимости экранов между Android и iOS.

Точкой входа служит класс App, который определяет стартовый экран по состоянию пользовательской сессии. При отсутствии авторизации отображается LoginPage, а после входа открывается AppShell с вкладками «Каталог», «Заказы» и «Профиль».

Таблица 23

Состав решения SportRent

Проект	Назначение
SportRent.DbTool	Сборка SQLite-базы из упорядоченных SQL-скриптов

SportRent.Data	Scaffolded EF Core сущности и DbContext
SportRent.Mobile	Кроссплатформенное приложение с экранами, сервисами и навигацией для мобильной и десктопной сборки

Таблица 24

Целевые платформы единой кодовой базы MAUI

Цель сборки	Назначение
net9.0-android	Мобильная версия приложения для Android
net9.0-ios	Мобильная версия приложения для iOS
net9.0-maccatalyst	Настольная сборка для macOS через Mac Catalyst
net9.0-windows10.0.19041.0	Десктопная сборка для Windows

ГЛАВА 3. РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

3.1 Реализация основных пользовательских сценариев

Сервис авторизации обращается к локальной базе данных, загружает демонстрационные учетные записи и выполняет проверку логина и пароля по таблице users с учетом роли пользователя. В учебной версии проекта учетные данные используются в демонстрационном виде, что упрощает проверку интерфейса и базы данных.

Сервис каталога формирует снимок данных для главного экрана, включающий список категорий, оборудование и сводные показатели по количеству позиций, пунктов проката и категорий. Для карточки инвентаря сервис дополнительно извлекает тарифы аренды, величину залога и сведения о доступности по точкам проката.

Наиболее важным с точки зрения прикладной логики является сценарий оформления аренды. В сервисе заказов реализована транзакционная последовательность действий: проверка остатка, уменьшение доступного количества, создание записи заказа, добавление позиции заказа и автоматическое создание платежа со статусом ожидания оплаты. Такой порядок обеспечивает согласованность локальных данных.

Подготовленные демонстрационные данные содержат 3 учетные записи, 1 тестовый заказ, 2 позицию(й) заказа и 1 платежную запись. Благодаря этому пользователь может показать на защите не только каталог, но и уже заполненную историю аренды.

3.2 Разработка интерфейса приложения

Интерфейс приложения оформлен в едином визуальном стиле и ориентирован на быстрое прохождение основных сценариев. На экранах используются крупные карточки, сводные метрики, фильтрация по категориям и визуально выделенные области для стоимости, залога и доступности.

Интерфейс мобильной версии ориентирован на вертикальный сценарий просмотра: блоки последовательно выстраиваются сверху вниз, а ключевые действия размещены в видимой зоне экрана, что упрощает сенсорное взаимодействие.

Поскольку проект использует единую XAML-разметку .NET MAUI, в дипломной пояснительной записке используются скриншоты Windows-сборки, отражающие те же экраны и пользовательские сценарии, что и мобильные цели Android и iOS.

Экран авторизации.

Первым для пользователя открывается экран входа, на котором размещены предзаполненные поля почты и пароля, а также список демонстрационных учетных записей. Такое решение уменьшает время подготовки к защите и позволяет быстро перейти к показу основных функций приложения.

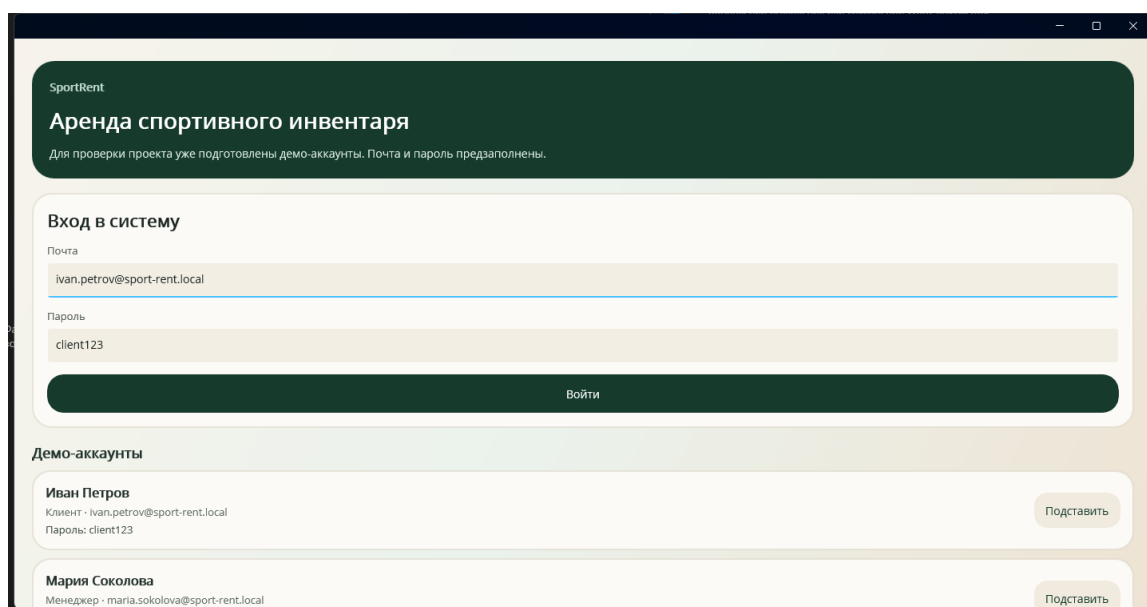


Рис. 2. Экран входа в систему с демонстрационными учетными записями

Главный экран каталога.

После успешной авторизации открывается вкладка каталога. На ней отображаются агрегированные показатели по количеству позиций, пунктов

проката и категорий, строка поиска, горизонтальный набор фильтров и карточки инвентаря с кратким описанием, стартовой ценой и залогом.

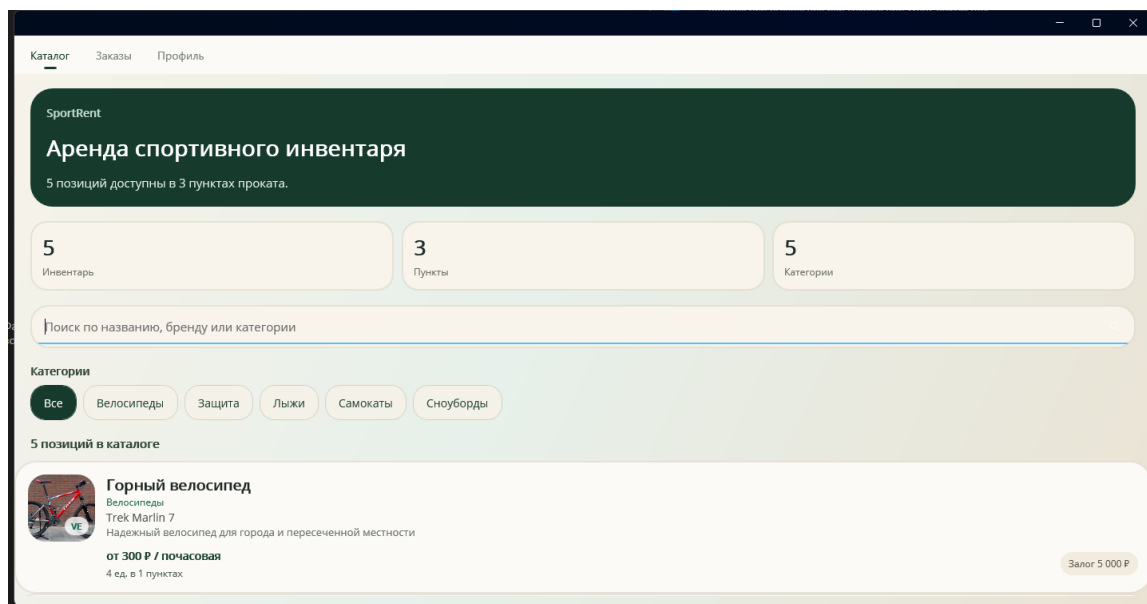


Рис. 3. Главный экран каталога спортивного инвентаря

Карточка инвентаря и переход к оформлению аренды.

При выборе позиции из каталога пользователь переходит на экран подробной карточки. Здесь собраны сведения о категории, бренде и модели, минимальной стоимости аренды, размере залога, доступности по пунктам проката и доступных тарифах. С этого экрана выполняется переход к отдельной форме оформления аренды, где пользователь выбирает тариф, пункт выдачи, дату начала, количество единиц и сразу видит итоговый расчет платежа.

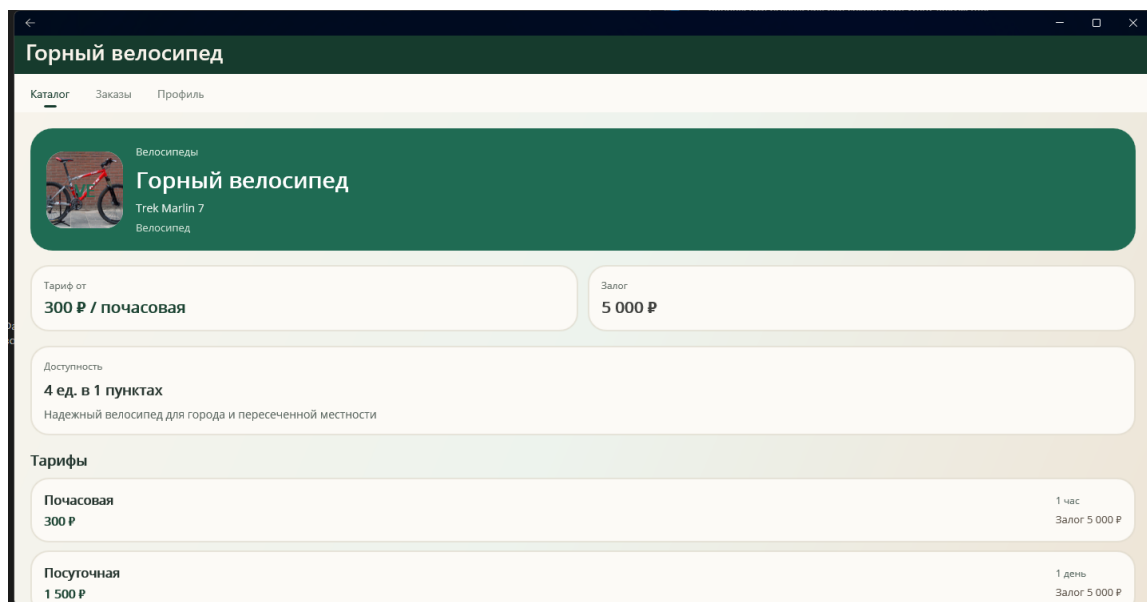


Рис. 4. Карточка выбранной позиции инвентаря

Экран истории заказов.

Раздел заказов показывает уже оформленные аренды и предназначен для проверки результатов транзакционного сценария. На экране видны статус заказа, сроки аренды, сумма аренды, размер залога, итоговый платеж и состояние оплаты, что позволяет наглядно проверить корректность записи данных в локальную базу.

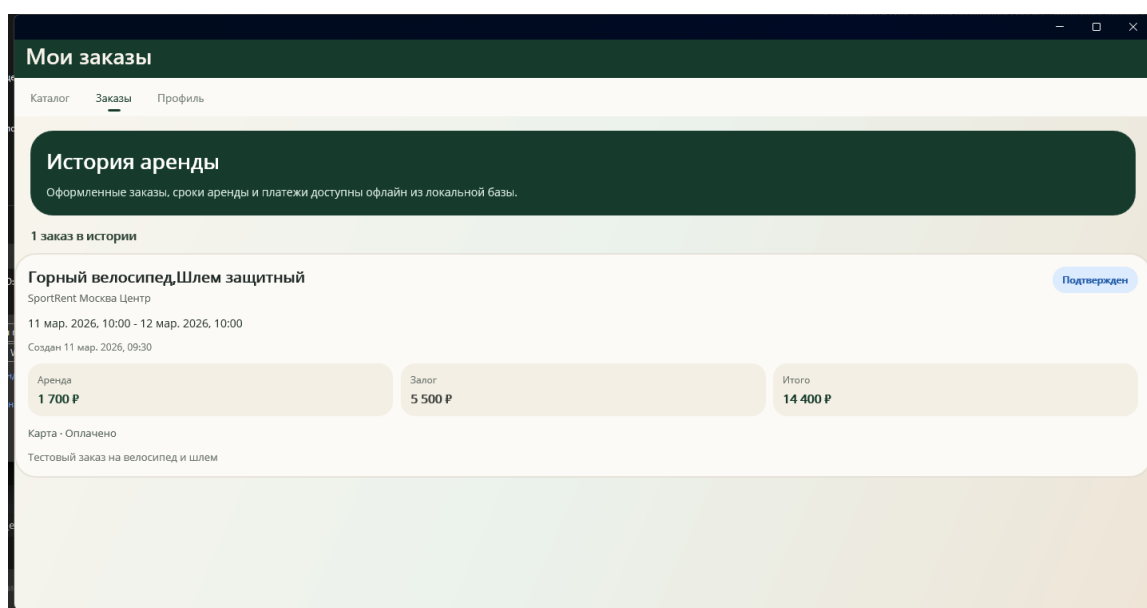


Рис. 5. Экран истории заказов пользователя

Экран профиля пользователя.

Вкладка профиля завершает пользовательский маршрут и содержит персональные данные, роль, контакты, дату регистрации и сводные показатели по заказам и оплатам. Наличие кнопки выхода из аккаунта подтверждает завершенность базового сценария работы с пользовательской сессией.

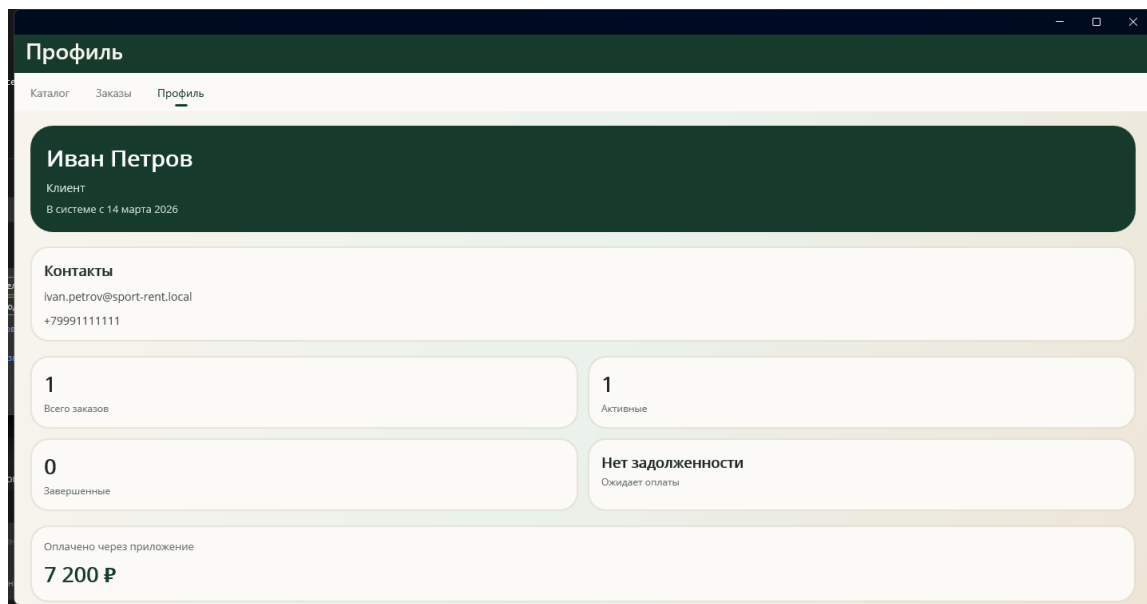


Рис. 6. Экран профиля пользователя

Представленные экраны показывают, что тема дипломного проекта соответствует фактически реализованному приложению: пользователь может войти в систему, выбрать инвентарь, ознакомиться с условиями аренды, посмотреть историю заказов и получить сведения о своем профиле.

3.3 Тестирование и проверка работоспособности

3.3.1 Сборка локальной базы данных

Для подготовки базы данных реализован отдельный консольный инструмент SportRent.DbTool. Он выполняет поиск SQL-файлов формата 000_name.sql в каталоге database, сортирует их по имени, создает временный файл SQLite и последовательно выполняет все найденные сценарии в рамках транзакции.

В проекте используются три канонических сценария: 001_recreate.sql, 002_schema.sql и 003_seed.sql. Первый сценарий отвечает за пересоздание структуры, второй - за формирование схемы, третий - за наполнение демонстрационными данными.

После успешного выполнения SQL-скриптов временный файл базы данных заменяет итоговый файл sportRent.db в ресурсах мобильного приложения. Данный подход снижает риск повреждения рабочей базы и делает процесс подготовки данных воспроизводимым.

3.3.2 Проверка пользовательских сценариев

Проект ориентирован на мобильные цели net9.0-android и net9.0-ios, однако для воспроизводимой локальной проверки, подготовки скриншотов и отладки в репозитории подтверждена сборка Windows-цели net9.0-windows10.0.19041.0. Общая XAML-разметка, ViewModel и сервисы позволяют рассматривать данную сборку как эквивалентный демонстрационный контур мобильной версии дипломного проекта.

В результате проверки установлено, что приложение корректно отображает 5 позиций каталога, сгруппированных по 5 категориям и доступных в 3 пунктах проката. История заказов показывает подготовленный тестовый заказ, а профиль пользователя агрегирует статистику по заказам и платежам.

Основной проверочный сценарий включает вход под демонстрационным пользователем, просмотр каталога, открытие карточки инвентаря, переход к истории заказов и просмотру личного кабинета. Этот набор действий показывает связность интерфейса, корректность навигации и согласованность локальных данных.

3.3.3 Ограничения текущей версии

Рассматриваемая реализация является учебной и не претендует на уровень промышленной системы. Это следует учитывать при оценке результатов и определении направлений дальнейшего развития проекта.

1. Авторизация демонстрационная, пароль хранится в seed-данных без криптографического хеширования.
2. Серверная часть и синхронизация с удаленной базой данных отсутствуют.
3. Регистрация новых пользователей и полноценная ролевая модель интерфейса не реализованы.
4. Реальная интеграция с платежной системой отсутствует.
5. Автоматизированные тесты в текущей версии не разработаны.
6. Практическая демонстрация в репозитории выполнена преимущественно на Windows-сборке, хотя кодовая база ориентирована на мобильные цели Android и iOS.

Несмотря на перечисленные ограничения, проект решает поставленные учебные задачи и демонстрирует полный маршрут обработки данных от структуры базы до экранов конечного пользователя.

ГЛАВА 4. РАСЧЕТ ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ ПРОЕКТА

4.1 Анализ затрат на разработку

Расчет экономической эффективности выполнен в учебных целях и носит оценочный характер. Он позволяет показать, какие трудозатраты и сопутствующие расходы могут сопровождать разработку подобного программного продукта.

В расчет включены основные этапы проектирования, программной реализации, тестирования, а также условные затраты на использование вычислительной техники. Оценка построена на трудоемкости работ, наблюдаемой в ходе разработки учебного проекта SportRent.

Таблица 25

Оценка затрат на разработку проекта

Статья затрат	Основание расчета	Сумма, руб.
Анализ и проектирование	48 ч x 350 руб./ч	16 800
Разработка базы данных и сервисов	92 ч x 350 руб./ч	32 200
Разработка пользовательского интерфейса	76 ч x 350 руб./ч	26 600
Тестировани е и отладка	40 ч x 320 руб./ч	12 800

Электроэнергия и амортизация ПК	Условный расчет	4 600
Итого		93 000

4.2 Оценка экономической эффективности

Для оценки эффекта рассмотрен типовой пункт проката, в котором часть операций менеджера переносится в приложение: просмотр каталога, уточнение доступности, расчет стоимости аренды и фиксация заказа. Это уменьшает время обслуживания одного клиента и снижает вероятность повторного ввода данных.

Показатели экономического эффекта основаны на условных, но реалистичных допущениях по количеству обращений и стоимости рабочего времени сотрудника. Такой подход соответствует учебному характеру проекта и позволяет определить ориентировочный срок окупаемости.

Таблица 26

Оценка ежемесячного экономического эффекта

Показатель	Значение
Среднее сокращение времени на оформление одной аренды	8 минут
Среднее количество заявок в месяц	180
Экономия рабочего времени в месяц	24 часа

Средняя стоимость часа работы сотрудника	350 руб.
Условная ежемесячная экономия	8 400 руб.
Дополнительный эффект от снижения ошибок и повторных операций	4 000 руб.
Совокупный экономический эффект в месяц	12 400 руб.
Ориентировочный срок окупаемости	около 7,5 месяцев

Полученные оценки показывают, что даже при учебном масштабе проекта приложение способно сократить операционные затраты пункта проката и окупить условные затраты на разработку в разумный срок. Следовательно, разработка SportRent является не только технически реализуемой, но и экономически целесообразной в модели демонстрационного внедрения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта разработано мобильное кроссплатформенное приложение SportRent для аренды спортивного инвентаря с локальной SQLite-базой данных.

В результате работы спроектирована нормализованная база данных, создан инструмент ее автоматической сборки из SQL-скриптов, выполнено scaffold-моделирование слоя данных и реализовано мобильное приложение на .NET MAUI. Пользователю доступны авторизация, каталог, карточка инвентаря, оформление аренды, история заказов и профиль.

Поставленная цель достигнута, так как тема дипломного проекта «Разработка мобильного приложения для аренды спортивного инвентаря» подтверждена фактической реализацией программного средства. Полученный результат может служить основой для дальнейшего развития проекта, включая добавление серверной части, улучшение безопасности, расширение ролей пользователей и интеграцию с внешними сервисами.

СПИСОК ЛИТЕРАТУРЫ

1. Методические указания к дипломному проектированию. Локальный документ «МУ к ДП.pdf».
2. Репозиторий SportRent. Файл README.md.
3. Репозиторий SportRent. Файл docs/explanatory-note-agent-context.json.
4. Репозиторий SportRent. Файл SportRent.Mobile/SportRent.Mobile.csproj.
5. Репозиторий SportRent. Исходный код проекта SportRent.DbTool, файл Program.cs.
6. Репозиторий SportRent. Скрипт SportRent.Data/scaffold.ps1.
7. Репозиторий SportRent. Исходный код сервиса авторизации SportRent.Mobile/Services/AuthenticationService.cs.
8. Репозиторий SportRent. Исходный код каталожного сервиса SportRent.Mobile/Services/SportRentCatalogService.cs.
9. Репозиторий SportRent. Исходный код сервиса заказов SportRent.Mobile/Services/OrdersService.cs.
10. Репозиторий SportRent. SQL-сценарий структуры базы данных database/002_schema.sql.
11. Репозиторий SportRent. SQL-сценарий начального заполнения database/003_seed.sql.
12. Microsoft Learn. Documentation for .NET MAUI.
13. Microsoft Learn. Documentation for Entity Framework Core.
14. SQLite Documentation.

Листинг кода инструмента сборки базы данных (Program.cs)

```

0001: using System.Text.RegularExpressions;
0002: using Microsoft.Data.Sqlite;
0003:
0004: internal static class Program
0005: {
0006:     private static readonly Regex ScriptFilePattern = new(
0007:         @"^\\d{3}_+\\.sql$",
0008:         RegexOptions.Compiled | RegexOptions.CultureInvariant |
RegexOptions.IgnoreCase);
0009:
0010:     private static int Main(string[] args)
0011:     {
0012:         try
0013:         {
0014:             BuildOptions options = ParseArguments(args);
0015:             string root = ResolveRoot(options.Root);
0016:             string scriptsDirectory = ResolvePath(root, options.ScriptsDirectory
?? "database");
0017:             string outputPath = ResolvePath(
0018:                 root,
0019:                 options.OutputPath ?? Path.Combine("SportRent.Mobile",
"Resources", "Raw", "Database", "sportRent.db"));
0020:             string[] scripts = GetScriptFiles(scriptsDirectory);
0021:
0022:             Console.WriteLine($"[INFO] Root: {root}");
0023:             Console.WriteLine($"[INFO] Scripts dir: {scriptsDirectory}");
0024:             Console.WriteLine($"[INFO] DB file: {outputPath}");
0025:             Console.WriteLine($"[INFO] Scripts found: {scripts.Length}");
0026:
0027:             BuildDatabase(outputPath, scripts);
0028:
0029:             Console.WriteLine("[SUCCESS] Database created.");
0030:             return 0;
0031:         }
0032:         catch (Exception ex)
0033:         {
0034:             Console.Error.WriteLine("[ERROR] Failed to build database.");
0035:             Console.Error.WriteLine(ex);
0036:             return 1;
0037:         }
0038:     }
0039:
0040:     private static BuildOptions ParseArguments(string[] args)
0041:     {
0042:         string? root = null;
0043:         string? scriptsDirectory = null;
0044:         string? outputPath = null;
0045:
0046:         for (int i = 0; i < args.Length; i++)
0047:         {
0048:             string arg = args[i];
0049:
0050:             if (arg is "--help" or "-h")
0051:             {
0052:                 PrintUsage();
0053:                 Environment.Exit(0);
0054:             }
0055:
0056:             string value = ReadValue(args, ref i);
0057:
0058:             switch (arg)

```

```

0059:         {
0060:             case "--root":
0061:                 root = value;
0062:                 break;
0063:             case "--scripts-dir":
0064:                 scriptsDirectory = value;
0065:                 break;
0066:             case "--output":
0067:                 outputPath = value;
0068:                 break;
0069:             default:
0070:                 throw new ArgumentException($"Unknown argument: {arg}");
0071:         }
0072:     }
0073:
0074:     return new BuildOptions(root, scriptsDirectory, outputPath);
0075: }
0076:
0077: private static string ReadValue(string[] args, ref int index)
0078: {
0079:     if (index + 1 >= args.Length)
0080:     {
0081:         throw new ArgumentException($"Expected a value after
0082: {args[index]}");
0083:     }
0084:     index++;
0085:     return args[index];
0086: }
0087:
0088: private static string ResolveRoot(string? rootArgument)
0089: {
0090:     if (!string.IsNullOrEmpty(rootArgument))
0091:     {
0092:         string root = Path.GetFullPath(rootArgument);
0093:
0094:         if (!Directory.Exists(root))
0095:         {
0096:             throw new DirectoryNotFoundException($"Root directory not found:
0097: {root}");
0098:         }
0099:         return root;
0100:     }
0101:
0102:     return FindProjectRoot();
0103: }
0104:
0105: private static string ResolvePath(string root, string path)
0106: {
0107:     return Path.GetFullPath(Path.IsPathRooted(path) ? path :
0108: Path.Combine(root, path));
0109: }
0110:
0111: private static string[] GetScriptFiles(string scriptsDirectory)
0112: {
0113:     if (!Directory.Exists(scriptsDirectory))
0114:     {
0115:         throw new DirectoryNotFoundException($"Scripts directory not found:
0116: {scriptsDirectory}");
0117:     }
0118:
0119:     string[] scripts = Directory
0120: .EnumerateFiles(scriptsDirectory, "*.sql",
0121: SearchOption.TopDirectoryOnly)
0122: .Where(path => ScriptFilePattern.IsMatch(Path.GetFileName(path)))
0123: .OrderBy(path => Path.GetFileName(path),
0124: StringComparer.OrdinalIgnoreCase)
0125: .ToArray();

```

```

0123:         if (scripts.Length == 0)
0124:         {
0125:             throw new InvalidOperationException(
0126:                 $"No numbered SQL scripts matching 000_name.sql were found in
{scriptsDirectory}");
0127:         }
0128:
0129:         return scripts;
0130:     }
0131:
0132:     private static void BuildDatabase(string outputPath, IReadOnlyList<string>
scripts)
0133:     {
0134:         string artifactsDirectory = Path.GetDirectoryName(outputPath)
0135:         ?? throw new InvalidOperationException($"Cannot determine output
directory for {outputPath}");
0136:
0137:         Directory.CreateDirectory(artifactsDirectory);
0138:
0139:         string tempPath = Path.Combine(
0140:             artifactsDirectory,
0141:             $"{Path.GetFileNameWithoutExtension(outputPath)}.{Guid.NewGuid():N}{Path.GetExtension(
outputPath)}");
0142:
0143:         try
0144:         {
0145:             using (SqliteConnection connection = OpenConnection(tempPath))
0146:             {
0147:                 using SqliteTransaction transaction =
connection.BeginTransaction();
0148:
0149:                 foreach (string scriptPath in scripts)
0150:                 {
0151:                     ExecuteScript(connection, transaction, scriptPath);
0152:                 }
0153:
0154:                 transaction.Commit();
0155:             }
0156:
0157:             ReplaceDatabase(tempPath, outputPath);
0158:         }
0159:         catch
0160:         {
0161:             TryDeleteFile(tempPath);
0162:             throw;
0163:         }
0164:     }
0165:
0166:     private static SqliteConnection OpenConnection(string dbPath)
0167:     {
0168:         string connectionString = new SqliteConnectionStringBuilder
0169:         {
0170:             DataSource = dbPath,
0171:             ForeignKeys = true,
0172:             Pooling = false
0173:         }.ToString();
0174:
0175:         var connection = new SqliteConnection(connectionString);
0176:         connection.Open();
0177:
0178:         return connection;
0179:     }
0180:
0181:     private static void ExecuteScript(
0182:         SqliteConnection connection,
0183:         SqliteTransaction transaction,
0184:         string scriptPath)
0185:     {
0186:         if (!File.Exists(scriptPath))

```

```

0187:         {
0188:             throw new FileNotFoundException("SQL script not found.",
scriptPath);
0189:         }
0190:
0191:         string sql = File.ReadAllText(scriptPath);
0192:
0193:         if (string.IsNullOrEmpty(sql))
0194:         {
0195:             throw new InvalidOperationException($"SQL script is empty:
{scriptPath}");
0196:         }
0197:
0198:         Console.WriteLine($"[INFO] Running {Path.GetFileName(scriptPath)}");
0199:
0200:         using SqliteCommand command = connection.CreateCommand();
0201:         command.Transaction = transaction;
0202:         command.CommandText = sql;
0203:         _ = command.ExecuteNonQuery();
0204:     }
0205:
0206:     private static void ReplaceDatabase(string tempPath, string outputPath)
0207:     {
0208:         if (File.Exists(outputPath))
0209:         {
0210:             Console.WriteLine("[INFO] Replacing existing database...");
0211:             RetryOnIOException(() => File.Delete(outputPath), $"Unable to delete
existing database: {outputPath}");
0212:         }
0213:
0214:         RetryOnIOException(() => File.Move(tempPath, outputPath), $"Unable to
place database at: {outputPath}");
0215:     }
0216:
0217:     private static void TryDeleteFile(string path)
0218:     {
0219:         try
0220:         {
0221:             if (File.Exists(path))
0222:             {
0223:                 File.Delete(path);
0224:             }
0225:         }
0226:         catch
0227:         {
0228:         }
0229:     }
0230:
0231:     private static void RetryOnIOException(Action action, string failureMessage)
0232:     {
0233:         const int maxAttempts = 10;
0234:
0235:         for (int attempt = 1; attempt <= maxAttempts; attempt++)
0236:         {
0237:             try
0238:             {
0239:                 action();
0240:                 return;
0241:             }
0242:             catch (IOException) when (attempt < maxAttempts)
0243:             {
0244:                 Thread.Sleep(200);
0245:             }
0246:         }
0247:
0248:         throw new IOException(failureMessage);
0249:     }
0250:
0251:     private static string FindProjectRoot()
0252:     {

```

```

0253:         foreach (string startPath in new[] { Directory.GetCurrentDirectory(),
AppContext.BaseDirectory })
0254:         {
0255:             var directory = new DirectoryInfo(startPath);
0256:             while (directory != null)
0257:             {
0258:                 if (Directory.Exists(Path.Combine(directory.FullName,
"database")))
0259:                 {
0260:                     return directory.FullName;
0261:                 }
0262:                 directory = directory.Parent;
0263:             }
0264:         }
0265:     }
0266:     }
0267:     }
0268:     throw new DirectoryNotFoundException("Project root with database folder
not found.");
0269: }
0270:
0271: private static void PrintUsage()
0272: {
0273:     Console.WriteLine("SportRent.DbTool usage:");
0274:     Console.WriteLine("  --root <path>           Optional project root.");
0275:     Console.WriteLine("  --scripts-dir <path> Optional SQL scripts
directory. Default: database");
0276:     Console.WriteLine("  --output <path>           Optional output DB path.
Default: SportRent.Mobile/Resources/Raw/Database/sportRent.db");
0277: }
0278:
0279: private sealed record BuildOptions(string? Root, string? ScriptsDirectory,
string? OutputPath);
0280: }

```


Листинг сервиса авторизации пользователя (AuthenticationService.cs)

```

0001: using Microsoft.Data.Sqlite;
0002: using SportRent.Mobile.Models;
0003:
0004: namespace SportRent.Mobile.Services;
0005:
0006: public sealed class AuthenticationService : SqliteServiceBase,
IAuthenticationService
0007: {
0008:     public AuthenticationService(ILocalDatabaseService localDatabaseService)
0009:         : base(localDatabaseService)
0010:     {
0011:     }
0012:
0013:     public async Task<IReadOnlyList<DemoAccount>>
GetDemoAccountsAsync(CancellationTokentoken = default)
0014:     {
0015:         await using SqliteConnection connection = await
OpenConnectionAsync(readOnly: true, cancellationTokentoken);
0016:
0017:         const string sql = ""
0018:             SELECT
0019:                 u.id,
0020:                 u.firstName,
0021:                 u.lastName,
0022:                 u.email,
0023:                 u.passwordHash,
0024:                 r.title AS roleTitle
0025:             FROM users u
0026:             INNER JOIN roles r ON r.id = u.idRole
0027:             ORDER BY u.idRole, u.id;
0028:             """;
0029:
0030:         await using SqliteCommand command = connection.CreateCommand();
0031:         command.CommandText = sql;
0032:
0033:         List<DemoAccount> accounts = [];
0034:         await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationTokentoken);
0035:
0036:         while (await reader.ReadAsync(cancellationTokentoken))
0037:         {
0038:             accounts.Add(new DemoAccount
0039:             {
0040:                 UserId = reader.GetInt32(reader.GetOrdinal("id")),
0041:                 DisplayName =
0042:                 $"{reader.GetString(reader.GetOrdinal("firstName"))}
{reader.GetString(reader.GetOrdinal("lastName"))}",
0043:                 RoleTitle = reader.GetString(reader.GetOrdinal("roleTitle")),
0044:                 Email = reader.GetString(reader.GetOrdinal("email")),
0045:                 Password = reader.GetString(reader.GetOrdinal("passwordHash"))
0046:             });
0047:         }
0048:         return accounts;
0049:     }
0050:
0051:     public async Task<UserSession?> SignInAsync(string email, string password,
CancellationTokentoken = default)
0052:     {

```

```

0053:         await using SqliteConnection connection = await
OpenConnectionAsync(readOnly: true, cancellationTokens);
0054:
0055:         const string sql = ""
0056:             SELECT
0057:                 u.id,
0058:                 u.firstName,
0059:                 u.lastName,
0060:                 u.email,
0061:                 u.phone,
0062:                 r.title AS roleTitle
0063:             FROM users u
0064:             INNER JOIN roles r ON r.id = u.idRole
0065:             WHERE lower(u.email) = lower($email)
0066:                 AND u.passwordHash = $password
0067:             LIMIT 1;
0068:             """;
0069:
0070:         await using SqliteCommand command = connection.CreateCommand();
0071:         command.CommandText = sql;
0072:         command.Parameters.AddWithValue("$email", email.Trim());
0073:         command.Parameters.AddWithValue("$password", password);
0074:
0075:         await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationTokens);
0076:         if (!await reader.ReadAsync(cancellationTokens))
0077:         {
0078:             return null;
0079:         }
0080:
0081:         return new UserSession
0082:         {
0083:             UserId = reader.GetInt32(reader.GetOrdinal("id")),
0084:             FirstName = reader.GetString(reader.GetOrdinal("firstName")),
0085:             LastName = reader.GetString(reader.GetOrdinal("lastName")),
0086:             Email = reader.GetString(reader.GetOrdinal("email")),
0087:             Phone = reader.GetString(reader.GetOrdinal("phone")),
0088:             RoleTitle = reader.GetString(reader.GetOrdinal("roleTitle"))
0089:         };
0090:     }
0091:
0092:     public async Task<UserProfile?> GetProfileAsync(int userId,
CancellationTokens cancellationTokens = default)
0093:     {
0094:         await using SqliteConnection connection = await
OpenConnectionAsync(readOnly: true, cancellationTokens);
0095:
0096:         const string sql = ""
0097:             SELECT
0098:                 u.id,
0099:                 u.firstName,
0100:                 u.lastName,
0101:                 u.email,
0102:                 u.phone,
0103:                 u.dateCreated,
0104:                 r.title AS roleTitle,
0105:                 COALESCE((
0106:                     SELECT COUNT(*)
0107:                     FROM rentOrders ro
0108:                     WHERE ro.idUser = u.id
0109:                 ), 0) AS totalOrders,
0110:                 COALESCE((
0111:                     SELECT COUNT(*)
0112:                     FROM rentOrders ro
0113:                     INNER JOIN orderStatuses os ON os.id = ro.idStatus
0114:                     WHERE ro.idUser = u.id
0115:                     AND os.title IN ('Создан', 'Подтвержден', 'В аренде')
0116:                 ), 0) AS activeOrders,
0117:                 COALESCE((
0118:                     SELECT COUNT(*)

```

```

0119:         FROM rentOrders ro
0120:         INNER JOIN orderStatuses os ON os.id = ro.idStatus
0121:         WHERE ro.idUser = u.id
0122:             AND os.title = 'Завершен'
0123:     ), 0) AS completedOrders,
0124:     COALESCE((
0125:         SELECT SUM(p.amount)
0126:         FROM payments p
0127:         INNER JOIN paymentStatuses ps ON ps.id = p.idStatus
0128:         INNER JOIN rentOrders ro ON ro.id = p.idOrder
0129:         WHERE ro.idUser = u.id
0130:             AND ps.title = 'Оплачено'
0131:     ), 0) AS totalPaidAmount,
0132:     COALESCE((
0133:         SELECT SUM(p.amount)
0134:         FROM payments p
0135:         INNER JOIN paymentStatuses ps ON ps.id = p.idStatus
0136:         INNER JOIN rentOrders ro ON ro.id = p.idOrder
0137:         WHERE ro.idUser = u.id
0138:             AND ps.title = 'Ожидает оплаты'
0139:     ), 0) AS outstandingAmount
0140: FROM users u
0141: INNER JOIN roles r ON r.id = u.idRole
0142: WHERE u.id = $userId
0143: LIMIT 1;
0144: """;
0145:
0146: await using SqliteCommand command = connection.CreateCommand();
0147: command.CommandText = sql;
0148: command.Parameters.AddWithValue("$userId", userId);
0149:
0150: await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationToken);
0151: if (!await reader.ReadAsync(cancellationToken))
0152: {
0153:     return null;
0154: }
0155:
0156: return new UserProfile
0157: {
0158:     UserId = reader.GetInt32(reader.GetOrdinal("id")),
0159:     FirstName = reader.GetString(reader.GetOrdinal("firstName")),
0160:     LastName = reader.GetString(reader.GetOrdinal("lastName")),
0161:     Email = reader.GetString(reader.GetOrdinal("email")),
0162:     Phone = reader.GetString(reader.GetOrdinal("phone")),
0163:     RoleTitle = reader.GetString(reader.GetOrdinal("roleTitle")),
0164:     DateCreated = ReadDateTime(reader, "dateCreated"),
0165:     TotalOrders = reader.GetInt32(reader.GetOrdinal("totalOrders")),
0166:     ActiveOrders = reader.GetInt32(reader.GetOrdinal("activeOrders")),
0167:     CompletedOrders =
reader.GetInt32(reader.GetOrdinal("completedOrders")),
0168:     TotalPaidAmount =
reader.GetInt32(reader.GetOrdinal("totalPaidAmount")),
0169:     OutstandingAmount =
reader.GetInt32(reader.GetOrdinal("outstandingAmount"))
0170: };
0171: }
0172: }

```

Листинг сервиса каталога спортивного инвентаря (SportRentCatalogService.cs)

```

0001: using Microsoft.Data.Sqlite;
0002: using SportRent.Mobile.Infrastructure;
0003: using SportRent.Mobile.Models;
0004:
0005: namespace SportRent.Mobile.Services;
0006:
0007: public sealed class SportRentCatalogService : SqliteServiceBase,
ISportRentCatalogService
0008: {
0009:     public SportRentCatalogService(ILocalDatabaseService localDatabaseService)
0010:     : base(localDatabaseService)
0011:     {
0012:     }
0013:
0014:     public async Task<CatalogSnapshot> GetCatalogAsync(CancellationToken
cancellationTok
0015:     {
0016:         await using SqliteConnection connection = await
OpenConnectionAsyn
0017:
0018:         IReadOnlyList<CatalogCategory> categories = await
LoadCategoriesAsyn
0019:         IReadOnlyList<CatalogEquipmentItem> equipment = await
LoadEquipmentAsyn
0020:         CatalogStats stats = await LoadStatsAsync(connection,
cancellationTok
0021:
0022:         return new CatalogSnapshot
0023:         {
0024:             Categories = categories,
0025:             Equipment = equipment,
0026:             Stats = stats
0027:         };
0028:     }
0029:
0030:     public async Task<EquipmentDetails?> GetEquipmentDetailsAsync(int
equipmentId, Canc
0031:     {
0032:         await using SqliteConnection connection = await
OpenConnectionAsyn
0033:
0034:         const string detailsSql = ""
0035:         SELECT
0036:             e.id,
0037:             e.idCategory,
0038:             e.title,
0039:             c.title AS categoryTitle,
0040:             b.title AS brandTitle,
0041:             et.title AS typeTitle,
0042:             e.model,
0043:             e.description,
0044:             (
0045:                 SELECT i.url
0046:                 FROM equipmentPhotos ep
0047:                 INNER JOIN images i ON i.id = ep.idImage
0048:                 WHERE ep.idEquipment = e.id
0049:                 ORDER BY ep.id
0050:                 LIMIT 1
0051:             ) AS imageUrl,

```

```

0052:         COALESCE((
0053:             SELECT MIN(er.price)
0054:             FROM equipmentRates er
0055:             WHERE er.idEquipment = e.id
0056:         ), 0) AS minPrice,
0057:         COALESCE((
0058:             SELECT rt.title
0059:             FROM equipmentRates er
0060:             INNER JOIN rentalTypes rt ON rt.id = er.idRentalType
0061:             WHERE er.idEquipment = e.id
0062:             ORDER BY er.price ASC, rt.unitHours ASC
0063:             LIMIT 1
0064:         ), '') AS minPriceType,
0065:         COALESCE((
0066:             SELECT MAX(er.deposit)
0067:             FROM equipmentRates er
0068:             WHERE er.idEquipment = e.id
0069:         ), 0) AS maxDeposit,
0070:         COALESCE((
0071:             SELECT SUM(rpe.availableQuantity)
0072:             FROM rentalPointEquipment rpe
0073:             WHERE rpe.idEquipment = e.id
0074:         ), 0) AS availableUnits,
0075:         COALESCE((
0076:             SELECT COUNT(DISTINCT rpe.idRentalPoint)
0077:             FROM rentalPointEquipment rpe
0078:             WHERE rpe.idEquipment = e.id
0079:             AND rpe.availableQuantity > 0
0080:         ), 0) AS rentalPointCount
0081:     FROM equipment e
0082:     INNER JOIN categories c ON c.id = e.idCategory
0083:     INNER JOIN brands b ON b.id = e.idBrand
0084:     INNER JOIN equipmentTypes et ON et.id = e.idType
0085:     WHERE e.id = $equipmentId;
0086:     """
0087:
0088:     await using SqliteCommand command = connection.CreateCommand();
0089:     command.CommandText = detailsSql;
0090:     command.Parameters.AddWithValue("$equipmentId", equipmentId);
0091:
0092:     await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationToken);
0093:     if (!await reader.ReadAsync(cancellationToken))
0094:     {
0095:         return null;
0096:     }
0097:
0098:     string categoryTitle =
reader.GetString(reader.GetOrdinal("categoryTitle"));
0099:     string typeTitle = reader.GetString(reader.GetOrdinal("typeTitle"));
0100:     (string accentColor, string accentSurfaceColor, string symbolText) =
EquipmentVisualIdentity.Resolve(categoryTitle, typeTitle);
0101:
0102:     IReadOnlyList<EquipmentRate> rates = await LoadRatesAsync(connection,
equipmentId, cancellationToken);
0103:     IReadOnlyList<RentalPointAvailability> rentalPoints = await
LoadRentalPointsAsync(connection, equipmentId, cancellationToken);
0104:
0105:     return new EquipmentDetails
0106:     {
0107:         Id = reader.GetInt32(reader.GetOrdinal("id")),
0108:         CategoryId = reader.GetInt32(reader.GetOrdinal("idCategory")),
0109:         Title = reader.GetString(reader.GetOrdinal("title")),
0110:         CategoryTitle = categoryTitle,
0111:         BrandTitle = reader.GetString(reader.GetOrdinal("brandTitle")),
0112:         TypeTitle = typeTitle,
0113:         Model = reader.IsDBNull(reader.GetOrdinal("model")) ? null :
reader.GetString(reader.GetOrdinal("model")),
0114:         Description = reader.IsDBNull(reader.GetOrdinal("description")) ?
null : reader.GetString(reader.GetOrdinal("description")),

```

```

0115:         imageUrl = reader.IsDBNull(reader.GetOrdinal("imageUrl")) ? null :
reader.GetString(reader.GetOrdinal("imageUrl")),
0116:         StartingPrice = reader.GetInt32(reader.GetOrdinal("minPrice")),
0117:         StartingRentalTypeTitle =
reader.GetString(reader.GetOrdinal("minPriceType")),
0118:         Deposit = reader.GetInt32(reader.GetOrdinal("maxDeposit")),
0119:         AvailableUnits =
reader.GetInt32(reader.GetOrdinal("availableUnits")),
0120:         RentalPointCount =
reader.GetInt32(reader.GetOrdinal("rentalPointCount")),
0121:         AccentColor = accentColor,
0122:         AccentSurfaceColor = accentSurfaceColor,
0123:         SymbolText = symbolText,
0124:         Rates = rates,
0125:         RentalPoints = rentalPoints
0126:     };
0127: }
0128:
0129: private static async Task<IReadOnlyList<CatalogCategory>>
LoadCategoriesAsync(SqliteConnection connection, CancellationToken cancellationToken)
0130: {
0131:     const string sql = ""
0132:         SELECT
0133:             c.id,
0134:             c.title,
0135:             COALESCE(c.description, '') AS description
0136:         FROM categories c
0137:         ORDER BY c.title;
0138:     """;
0139:
0140:     await using SqliteCommand command = connection.CreateCommand();
0141:     command.CommandText = sql;
0142:
0143:     List<CatalogCategory> categories = [];
0144:     await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationToken);
0145:
0146:     while (await reader.ReadAsync(cancellationToken))
0147:     {
0148:         categories.Add(new CatalogCategory
0149:         {
0150:             Id = reader.GetInt32(reader.GetOrdinal("id")),
0151:             Title = reader.GetString(reader.GetOrdinal("title")),
0152:             Description = reader.GetString(reader.GetOrdinal("description"))
0153:         });
0154:     }
0155:
0156:     return categories;
0157: }
0158:
0159: private static async Task<IReadOnlyList<CatalogEquipmentItem>>
LoadEquipmentAsync(SqliteConnection connection, CancellationToken cancellationToken)
0160: {
0161:     const string sql = ""
0162:         SELECT
0163:             e.id,
0164:             e.idCategory,
0165:             e.title,
0166:             c.title AS categoryTitle,
0167:             b.title AS brandTitle,
0168:             et.title AS typeTitle,
0169:             e.model,
0170:             e.description,
0171:             (
0172:                 SELECT i.url
0173:                 FROM equipmentPhotos ep
0174:                 INNER JOIN images i ON i.id = ep.idImage
0175:                 WHERE ep.idEquipment = e.id
0176:                 ORDER BY ep.id
0177:                 LIMIT 1

```

```

0178:         ) AS imageUrl,
0179:         COALESCE((
0180:             SELECT MIN(er.price)
0181:             FROM equipmentRates er
0182:             WHERE er.idEquipment = e.id
0183:         ), 0) AS minPrice,
0184:         COALESCE((
0185:             SELECT rt.title
0186:             FROM equipmentRates er
0187:             INNER JOIN rentalTypes rt ON rt.id = er.idRentalType
0188:             WHERE er.idEquipment = e.id
0189:             ORDER BY er.price ASC, rt.unitHours ASC
0190:             LIMIT 1
0191:         ), '') AS minPriceType,
0192:         COALESCE((
0193:             SELECT MAX(er.deposit)
0194:             FROM equipmentRates er
0195:             WHERE er.idEquipment = e.id
0196:         ), 0) AS maxDeposit,
0197:         COALESCE((
0198:             SELECT SUM(rpe.availableQuantity)
0199:             FROM rentalPointEquipment rpe
0200:             WHERE rpe.idEquipment = e.id
0201:         ), 0) AS availableUnits,
0202:         COALESCE((
0203:             SELECT COUNT(DISTINCT rpe.idRentalPoint)
0204:             FROM rentalPointEquipment rpe
0205:             WHERE rpe.idEquipment = e.id
0206:             AND rpe.availableQuantity > 0
0207:         ), 0) AS rentalPointCount
0208:     FROM equipment e
0209:     INNER JOIN categories c ON c.id = e.idCategory
0210:     INNER JOIN brands b ON b.id = e.idBrand
0211:     INNER JOIN equipmentTypes et ON et.id = e.idType
0212:     ORDER BY c.title, e.title;
0213:     """
0214:
0215:     await using SqliteCommand command = connection.CreateCommand();
0216:     command.CommandText = sql;
0217:
0218:     List<CatalogEquipmentItem> items = [];
0219:     await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationToken);
0220:
0221:     while (await reader.ReadAsync(cancellationToken))
0222:     {
0223:         string categoryTitle =
reader.GetString(reader.GetOrdinal("categoryTitle"));
0224:         string typeTitle = reader.GetString(reader.GetOrdinal("typeTitle"));
0225:         (string accentColor, string accentSurfaceColor, string symbolText) =
EquipmentVisualIdentity.Resolve(categoryTitle, typeTitle);
0226:
0227:         items.Add(new CatalogEquipmentItem
0228:         {
0229:             Id = reader.GetInt32(reader.GetOrdinal("id")),
0230:             CategoryId = reader.GetInt32(reader.GetOrdinal("idCategory")),
0231:             Title = reader.GetString(reader.GetOrdinal("title")),
0232:             CategoryTitle = categoryTitle,
0233:             BrandTitle = reader.GetString(reader.GetOrdinal("brandTitle")),
0234:             TypeTitle = typeTitle,
0235:             Model = reader.IsDBNull(reader.GetOrdinal("model")) ? null :
reader.GetString(reader.GetOrdinal("model")),
0236:             Description = reader.IsDBNull(reader.GetOrdinal("description"))
? null : reader.GetString(reader.GetOrdinal("description")),
0237:             ImageUrl = reader.IsDBNull(reader.GetOrdinal("imageUrl")) ? null
: reader.GetString(reader.GetOrdinal("imageUrl")),
0238:             StartingPrice = reader.GetInt32(reader.GetOrdinal("minPrice")),
0239:             StartingRentalTypeTitle =
reader.GetString(reader.GetOrdinal("minPriceType")),
0240:             Deposit = reader.GetInt32(reader.GetOrdinal("maxDeposit")),

```

```

0241:             AvailableUnits =
reader.GetInt32(reader.GetOrdinal("availableUnits")),
0242:             RentalPointCount =
reader.GetInt32(reader.GetOrdinal("rentalPointCount")),
0243:             AccentColor = accentColor,
0244:             AccentSurfaceColor = accentSurfaceColor,
0245:             SymbolText = symbolText
0246:         });
0247:     }
0248:
0249:     return items;
0250: }
0251:
0252: private static async Task<CatalogStats> LoadStatsAsync(SqliteConnection
connection, Cancellation token cancellationToken)
0253: {
0254:     const string sql = ""
0255:         SELECT
0256:             (SELECT COUNT(*) FROM equipment) AS totalEquipment,
0257:             (SELECT COUNT(*) FROM rentalPoints) AS totalRentalPoints,
0258:             (SELECT COUNT(*) FROM categories) AS totalCategories;
0259:         ""
0260:
0261:     await using SqliteCommand command = connection.CreateCommand();
0262:     command.CommandText = sql;
0263:
0264:     await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationToken);
0265:     await reader.ReadAsync(cancellationToken);
0266:
0267:     return new CatalogStats
0268:     {
0269:         TotalEquipment =
reader.GetInt32(reader.GetOrdinal("totalEquipment")),
0270:         TotalRentalPoints =
reader.GetInt32(reader.GetOrdinal("totalRentalPoints")),
0271:         TotalCategories =
reader.GetInt32(reader.GetOrdinal("totalCategories"))
0272:     };
0273: }
0274:
0275: private static async Task<IReadOnlyList<EquipmentRate>>
LoadRatesAsync(SqliteConnection connection, int equipmentId, Cancellation token
cancellationToken)
0276: {
0277:     const string sql = ""
0278:         SELECT
0279:             rt.title,
0280:             rt.code,
0281:             rt.unitHours,
0282:             er.price,
0283:             er.deposit
0284:         FROM equipmentRates er
0285:         INNER JOIN rentalTypes rt ON rt.id = er.idRentalType
0286:         WHERE er.idEquipment = $equipmentId
0287:         ORDER BY rt.unitHours;
0288:         ""
0289:
0290:     await using SqliteCommand command = connection.CreateCommand();
0291:     command.CommandText = sql;
0292:     command.Parameters.AddWithValue("$equipmentId", equipmentId);
0293:
0294:     List<EquipmentRate> rates = [];
0295:     await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationToken);
0296:
0297:     while (await reader.ReadAsync(cancellationToken))
0298:     {
0299:         rates.Add(new EquipmentRate
0300:         {

```



```

0301:         Title = reader.GetString(reader.GetOrdinal("title")),
0302:         Code = reader.GetString(reader.GetOrdinal("code")),
0303:         UnitHours = reader.GetInt32(reader.GetOrdinal("unitHours")),
0304:         Price = reader.GetInt32(reader.GetOrdinal("price")),
0305:         Deposit = reader.GetInt32(reader.GetOrdinal("deposit"))
0306:     });
0307: }
0308:
0309:     return rates;
0310: }
0311:
0312:     private static async Task<IReadOnlyList<RentalPointAvailability>>
LoadRentalPointsAsync(SqliteConnection connection, int equipmentId, CancellationToken
cancellationToken)
0313:     {
0314:         const string sql = ""
0315:             SELECT
0316:                 rpe.id,
0317:                 rpe.idRentalPoint,
0318:                 rp.name AS rentalPointName,
0319:                 rp.address,
0320:                 rp.phone,
0321:                 ec.title AS conditionTitle,
0322:                 es.title AS sizeTitle,
0323:                 rpe.availableQuantity,
0324:                 rpe.totalQuantity
0325:             FROM rentalPointEquipment rpe
0326:             INNER JOIN rentalPoints rp ON rp.id = rpe.idRentalPoint
0327:             INNER JOIN equipmentConditions ec ON ec.id =
rpe.idEquipmentCondition
0328:             INNER JOIN equipmentSizes es ON es.id = rpe.idSize
0329:             WHERE rpe.idEquipment = $equipmentId
0330:             ORDER BY rpe.availableQuantity DESC, rp.name, es.title;
0331:         """;
0332:
0333:         await using SqliteCommand command = connection.CreateCommand();
0334:         command.CommandText = sql;
0335:         command.Parameters.AddWithValue("$equipmentId", equipmentId);
0336:
0337:         List<RentalPointAvailability> rentalPoints = [];
0338:         await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationToken);
0339:
0340:         while (await reader.ReadAsync(cancellationToken))
0341:         {
0342:             rentalPoints.Add(new RentalPointAvailability
0343:             {
0344:                 Id = reader.GetInt32(reader.GetOrdinal("id")),
0345:                 RentalPointId =
reader.GetInt32(reader.GetOrdinal("idRentalPoint")),
0346:                 RentalPointName =
reader.GetString(reader.GetOrdinal("rentalPointName")),
0347:                 Address = reader.GetString(reader.GetOrdinal("address")),
0348:                 Phone = reader.IsDBNull(reader.GetOrdinal("phone")) ? null :
reader.GetString(reader.GetOrdinal("phone")),
0349:                 ConditionTitle =
reader.GetString(reader.GetOrdinal("conditionTitle")),
0350:                 SizeTitle = reader.GetString(reader.GetOrdinal("sizeTitle")),
0351:                 AvailableQuantity =
reader.GetInt32(reader.GetOrdinal("availableQuantity")),
0352:                 TotalQuantity =
reader.GetInt32(reader.GetOrdinal("totalQuantity"))
0353:             });
0354:         }
0355:
0356:         return rentalPoints;
0357:     }
0358:
0359: }

```


Листинг сервиса оформления и чтения заказов (OrdersService.cs)

```

0001: using Microsoft.Data.Sqlite;
0002: using SportRent.Mobile.Models;
0003:
0004: namespace SportRent.Mobile.Services;
0005:
0006: public sealed class OrdersService : SqliteServiceBase, IOrdersService
0007: {
0008:     public OrdersService(ILocalDatabaseService localDatabaseService)
0009:         : base(localDatabaseService)
0010:     {
0011:     }
0012:
0013:     public async Task<IReadOnlyList<UserOrder>> GetOrdersAsync(int userId,
CancellationToken cancellationToken = default)
0014:     {
0015:         await using SqliteConnection connection = await
OpenConnectionAsync(readOnly: true, cancellationToken);
0016:
0017:         const string sql = """
0018:             SELECT
0019:                 ro.id,
0020:                 os.title AS statusTitle,
0021:                 rpIssue.name AS issuePointName,
0022:                 rpReturn.name AS returnPointName,
0023:                 ro.description,
0024:                 ro.dateCreated,
0025:                 ro.dateStart,
0026:                 ro.dateEnd,
0027:                 ro.amount,
0028:                 ro.depositAmount,
0029:                 COALESCE(SUM(p.amount), ro.amount + ro.depositAmount) AS
totalPaymentAmount,
0030:                 MAX(pm.title) AS paymentMethodTitle,
0031:                 MAX(ps.title) AS paymentStatusTitle,
0032:                 COALESCE(GROUP_CONCAT(DISTINCT e.title), 'Инвентарь не указан')
AS equipmentSummary
0033:             FROM rentOrders ro
0034:             INNER JOIN orderStatuses os ON os.id = ro.idStatus
0035:             INNER JOIN rentalPoints rpIssue ON rpIssue.id =
ro.idRentalPointIssue
0036:             LEFT JOIN rentalPoints rpReturn ON rpReturn.id =
ro.idRentalPointReturn
0037:             LEFT JOIN orderItems oi ON oi.idOrder = ro.id
0038:             LEFT JOIN rentalPointEquipment rpe ON rpe.id =
oi.idRentalPointEquipment
0039:             LEFT JOIN equipment e ON e.id = rpe.idEquipment
0040:             LEFT JOIN payments p ON p.idOrder = ro.id
0041:             LEFT JOIN paymentMethods pm ON pm.id = p.idPaymentMethod
0042:             LEFT JOIN paymentStatuses ps ON ps.id = p.idStatus
0043:             WHERE ro.idUser = $userId
0044:             GROUP BY
0045:                 ro.id,
0046:                 os.title,
0047:                 rpIssue.name,
0048:                 rpReturn.name,
0049:                 ro.description,
0050:                 ro.dateCreated,
0051:                 ro.dateStart,
0052:                 ro.dateEnd,
0053:                 ro.amount,
0054:                 ro.depositAmount

```

```

0055:         ORDER BY ro.dateCreated DESC, ro.id DESC;
0056:         """;
0057:
0058:         await using SqliteCommand command = connection.CreateCommand();
0059:         command.CommandText = sql;
0060:         command.Parameters.AddWithValue("$userId", userId);
0061:
0062:         List<UserOrder> orders = [];
0063:         await using SqliteDataReader reader = await
command.ExecuteReaderAsync(cancellationTokens);
0064:
0065:         while (await reader.ReadAsync(cancellationTokens))
0066:         {
0067:             orders.Add(new UserOrder
0068:             {
0069:                 Id = reader.GetInt32(reader.GetOrdinal("id")),
0070:                 StatusTitle =
reader.GetString(reader.GetOrdinal("statusTitle")),
0071:                 IssuePointName =
reader.GetString(reader.GetOrdinal("issuePointName")),
0072:                 ReturnPointName =
reader.IsDBNull(reader.GetOrdinal("returnPointName")) ? null :
reader.GetString(reader.GetOrdinal("returnPointName")),
0073:                 EquipmentSummary =
reader.GetString(reader.GetOrdinal("equipmentSummary")),
0074:                 Description = reader.IsDBNull(reader.GetOrdinal("description"))
? null : reader.GetString(reader.GetOrdinal("description")),
0075:                 DateCreated = ReadDateTime(reader, "dateCreated"),
0076:                 DateStart = ReadDateTime(reader, "dateStart"),
0077:                 DateEnd = ReadDateTime(reader, "dateEnd"),
0078:                 Amount = reader.GetInt32(reader.GetOrdinal("amount")),
0079:                 DepositAmount =
reader.GetInt32(reader.GetOrdinal("depositAmount")),
0080:                 TotalPaymentAmount =
reader.GetInt32(reader.GetOrdinal("totalPaymentAmount")),
0081:                 PaymentMethodTitle =
reader.IsDBNull(reader.GetOrdinal("paymentMethodTitle")) ? null :
reader.GetString(reader.GetOrdinal("paymentMethodTitle")),
0082:                 PaymentStatusTitle =
reader.IsDBNull(reader.GetOrdinal("paymentStatusTitle")) ? null :
reader.GetString(reader.GetOrdinal("paymentStatusTitle"))
0083:             });
0084:         }
0085:
0086:         return orders;
0087:     }
0088:
0089:     public async Task<int> CreateOrderAsync(CreateOrderRequest request,
CancellationTokens cancellationTokens = default)
0090:     {
0091:         ArgumentNullException.ThrowIfNull(request);
0092:
0093:         await using SqliteConnection connection = await
OpenConnectionAsync(readOnly: false, cancellationTokens);
0094:         await using SqliteTransaction transaction = (SqliteTransaction)await
connection.BeginTransactionAsync(cancellationTokens);
0095:
0096:         SqliteCommand stockCommand = connection.CreateCommand();
0097:         stockCommand.Transaction = transaction;
0098:         stockCommand.CommandText = ""
0099:             SELECT
0100:                 rpe.idRentalPoint,
0101:                 rpe.availableQuantity
0102:             FROM rentalPointEquipment rpe
0103:             WHERE rpe.id = $rentalPointEquipmentId
0104:             LIMIT 1;
0105:             """;
0106:         stockCommand.Parameters.AddWithValue("$rentalPointEquipmentId",
request.RentalPointEquipmentId);
0107:

```

```

0108:         int issuePointId;
0109:         int availableQuantity;
0110:
0111:         await using (SqliteDataReader stockReader = await
stockCommand.ExecuteReaderAsync(cancellationToken))
0112:         {
0113:             if (!await stockReader.ReadAsync(cancellationToken))
0114:             {
0115:                 throw new InvalidOperationException("Выбранный остаток инвентаря
не найден.");
0116:             }
0117:
0118:             issuePointId =
stockReader.GetInt32(stockReader.GetOrdinal("idRentalPoint"));
0119:             availableQuantity =
stockReader.GetInt32(stockReader.GetOrdinal("availableQuantity"));
0120:         }
0121:
0122:         if (availableQuantity < request.Quantity)
0123:         {
0124:             throw new InvalidOperationException("Недостаточно свободного
инвентаря для оформления заказа.");
0125:         }
0126:
0127:         int orderStatusId = await GetLookupIdAsync(connection, transaction,
"orderStatuses", "Создан", cancellationToken);
0128:         int paymentMethodId = await GetLookupIdAsync(connection, transaction,
"paymentMethods", "Онлайн", cancellationToken);
0129:         int paymentStatusId = await GetLookupIdAsync(connection, transaction,
"paymentStatuses", "Ожидает оплаты", cancellationToken);
0130:
0131:         int rentalAmount = checked(request.PricePerUnit * request.PeriodCount *
request.Quantity);
0132:         int depositAmount = checked(request.DepositPerUnit * request.Quantity);
0133:         DateTime endAt = request.StartAt.AddHours(request.UnitHours *
request.PeriodCount);
0134:         string description = string.IsNullOrEmpty(request.Description)
0135:             ? $"Заказ на {request.EquipmentTitle}"
0136:             : request.Description.Trim();
0137:
0138:         SqliteCommand updateStockCommand = connection.CreateCommand();
0139:         updateStockCommand.Transaction = transaction;
0140:         updateStockCommand.CommandText = ""
0141:             UPDATE rentalPointEquipment
0142:             SET availableQuantity = availableQuantity - $quantity
0143:             WHERE id = $rentalPointEquipmentId
0144:             AND availableQuantity >= $quantity;
0145:             "";
0146:         updateStockCommand.Parameters.AddWithValue("$quantity",
request.Quantity);
0147:         updateStockCommand.Parameters.AddWithValue("$rentalPointEquipmentId",
request.RentalPointEquipmentId);
0148:
0149:         int updatedRows = await
updateStockCommand.ExecuteNonQueryAsync(cancellationToken);
0150:         if (updatedRows == 0)
0151:         {
0152:             throw new InvalidOperationException("Свободный остаток изменился.
Обновите карточку и попробуйте снова.");
0153:         }
0154:
0155:         SqliteCommand orderCommand = connection.CreateCommand();
0156:         orderCommand.Transaction = transaction;
0157:         orderCommand.CommandText = ""
0158:             INSERT INTO rentOrders (
0159:                 idUser,
0160:                 idStatus,
0161:                 idRentalPointIssue,
0162:                 idRentalPointReturn,
0163:                 dateStart,

```

```

0164:         dateEnd,
0165:         amount,
0166:         depositAmount,
0167:         description
0168:     )
0169:     VALUES (
0170:         $userId,
0171:         $statusId,
0172:         $issuePointId,
0173:         $returnPointId,
0174:         $dateStart,
0175:         $dateEnd,
0176:         $amount,
0177:         $depositAmount,
0178:         $description
0179:     );
0180:
0181:     SELECT last_insert_rowid();
0182:     "";
0183:     orderCommand.Parameters.AddWithValue("$userId", request.UserId);
0184:     orderCommand.Parameters.AddWithValue("$statusId", orderStatusId);
0185:     orderCommand.Parameters.AddWithValue("$issuePointId", issuePointId);
0186:     orderCommand.Parameters.AddWithValue("$returnPointId", issuePointId);
0187:     orderCommand.Parameters.AddWithValue("$dateStart",
request.StartAt.ToString("yyyy-MM-dd HH:mm:ss"));
0188:     orderCommand.Parameters.AddWithValue("$dateEnd", endAt.ToString("yyyy-
MM-dd HH:mm:ss"));
0189:     orderCommand.Parameters.AddWithValue("$amount", rentalAmount);
0190:     orderCommand.Parameters.AddWithValue("$depositAmount", depositAmount);
0191:     orderCommand.Parameters.AddWithValue("$description", description);
0192:
0193:     long orderIdRaw = (long)await
orderCommand.ExecuteScalarAsync(cancellationToken)
0194:     ?? throw new InvalidOperationException("Не удалось создать
заказ.");
0195:     int orderId = checked((int)orderIdRaw);
0196:
0197:     SqliteCommand orderItemCommand = connection.CreateCommand();
0198:     orderItemCommand.Transaction = transaction;
0199:     orderItemCommand.CommandText = ""
0200:         INSERT INTO orderItems (
0201:             idOrder,
0202:             idRentalPointEquipment,
0203:             quantity,
0204:             pricePerUnit,
0205:             amount
0206:         )
0207:         VALUES (
0208:             $orderId,
0209:             $rentalPointEquipmentId,
0210:             $quantity,
0211:             $pricePerUnit,
0212:             $amount
0213:         );
0214:     "";
0215:     orderItemCommand.Parameters.AddWithValue("$orderId", orderId);
0216:     orderItemCommand.Parameters.AddWithValue("$rentalPointEquipmentId",
request.RentalPointEquipmentId);
0217:     orderItemCommand.Parameters.AddWithValue("$quantity", request.Quantity);
0218:     orderItemCommand.Parameters.AddWithValue("$pricePerUnit",
request.PricePerUnit * request.PeriodCount);
0219:     orderItemCommand.Parameters.AddWithValue("$amount", rentalAmount);
0220:     await orderItemCommand.ExecuteNonQueryAsync(cancellationToken);
0221:
0222:     SqliteCommand paymentCommand = connection.CreateCommand();
0223:     paymentCommand.Transaction = transaction;
0224:     paymentCommand.CommandText = ""
0225:         INSERT INTO payments (
0226:             idOrder,
0227:             idPaymentMethod,

```

```

0228:             idStatus,
0229:             amount
0230:         )
0231:         VALUES (
0232:             $orderId,
0233:             $paymentMethodId,
0234:             $paymentStatusId,
0235:             $amount
0236:         );
0237:         "";
0238:         paymentCommand.Parameters.AddWithValue("$orderId", orderId);
0239:         paymentCommand.Parameters.AddWithValue("$paymentMethodId",
paymentMethodId);
0240:         paymentCommand.Parameters.AddWithValue("$paymentStatusId",
paymentStatusId);
0241:         paymentCommand.Parameters.AddWithValue("$amount", rentalAmount +
depositAmount);
0242:         await paymentCommand.ExecuteNonQueryAsync(cancellationToken);
0243:
0244:         await transaction.CommitAsync(cancellationToken);
0245:         return orderId;
0246:     }
0247:
0248:     private static async Task<int> GetLookupIdAsync(
0249:         SqliteConnection connection,
0250:         SqliteTransaction transaction,
0251:         string tableName,
0252:         string title,
0253:         CancellationToken cancellationToken)
0254:     {
0255:         SqliteCommand command = connection.CreateCommand();
0256:         command.Transaction = transaction;
0257:         command.CommandText = $"SELECT id FROM {tableName} WHERE title = $title
LIMIT 1;";
0258:         command.Parameters.AddWithValue("$title", title);
0259:
0260:         object? result = await command.ExecuteScalarAsync(cancellationToken);
0261:         if (result is null)
0262:         {
0263:             throw new InvalidOperationException($"Не найдено значение
справочника '{title}' в таблице {tableName}.");
0264:         }
0265:
0266:         return Convert.ToInt32(result);
0267:     }
0268: }

```